

SQL  $\Rightarrow$  Structured Query Language

\* Paper Information  $\rightarrow$  digital Storage  $\Rightarrow$  DB

\* To access the DB (or) To see the DB  
we must be want a DBMS.

DBMS  $\xrightarrow{\text{SQL}}$  DB

RDB  $\Rightarrow$  Relational DB  $\Rightarrow$  RDBMS  $\Rightarrow$  Language SQL  
NOSQL  
Hierarchical  
Network  
 $\downarrow$   
Oracle, MySQL, PostgreSQL, SQL Server

Types of DB:

SQL vs NO SQL

Data  $\Rightarrow$  Structured data (Text, no, Date)  
unstructured data (Social Media)  
Semi structured data (Email)

Email  
Semi structure { Sender  
Receiver  
date } structure  
Any kind of Information } unstructure

Relational

Non-Relational

SQL, structured

NO-SQL, Not Only SQL

Only Tables

{ document, graph, key value }

format.

$\downarrow$   
(Data in) NO SQL

## MySQL download

chrome  $\Rightarrow$  MySQL download  $\Rightarrow$  MySQL download community

No  $\Leftarrow$  Higher  
Thanks

$\Leftarrow$  windows  $\Leftarrow$  MySQL Installer

$\Downarrow$  full  $\Rightarrow$  Password  $\Rightarrow$  Next  $\Rightarrow$  Work Bench

Schema  $\Rightarrow$  Database

## SQL commands (5 types)

DDL  $\Rightarrow$  Data definitional Language

DML  $\Rightarrow$  Data Manipulation Language

TCL

DCL  $\Rightarrow$  Data control

DQL  $\Rightarrow$  Data Query Language

DDL (Table)

Table create

ALTER

DROP

TRUNCATE

DML  $\Rightarrow$  insert, delete, update

DQL  $\Rightarrow$  Select

DCL  $\Rightarrow$  Grant, Revoke

TCL  $\Rightarrow$  Roll Back, commit, Savepoint

## 1) DB creation

create schema dummy

create table teacher (

name varchar(100),

age int,

Salary int,

department varchar(100),

)

## 2). ALTER command

To add new element into the same DB.

alter table teacher add column degree <sup>new</sup>  
Varchar (100),

table name

How change department → dept  
alter table teacher 'change' department dept <sup>old name</sup>  
Varchar (100). <sup>new</sup>

## To change data Type

alter table teacher change age age Varchar (100)  
same same different datatype

## Drop table

Drop teacher table ⇒ delete the whole table

## Truncate command

truncate table teacher



Deletes the data from the table

## DQL ⇒ Select command (To view data)

use dummy

create table student (

name Varchar (100),

age int,

Department varchar (100),

)

}  
Table creation  
commands

To retrieve data from that Table

For this purpose we use DDL  
(Select commands)

Select \* from student  
everything

Inserting data into DB

insert into student Values ('Hilini', 18, 'CSE'),  
" " " ("Anun", 19, 'CSE'),  
" " " ("Afreen", 18, 'CSE'),

To update Anun's age 19 to 20

update student ~~age~~ set age = 20  
Keyword

ans: ella age ub 20 nu change

update student set department = "computer"  
where department = "CSE"  
Key need  
condition old

update student set age = 20 where  
name = "Anun"

Delete command

delete from student ⇒ deletes whole

delete from student where age = 18

↓  
ans: Engala 18 age varuko ellam delete

DML ⇒ Insert, update, delete

## Aggregate functions

Sum(), max(), min(), Avg(), count()

use dummy;

create table student {

student\_name Varchar(100),

student\_mark int,

department Varchar(100),

};

insert into student Values ("Praveen", 1, "CSE");

insert into student Values ("malini", 2, "CSE");

insert into student Values ("Dhano", 3, "ECE");

" " " " ("Arun", 4, "ECE");

select \* from student  $\Rightarrow$  whole table

select student\_mark from student

ans: student\_mark

1 :  
2 :  
3 :  
4 : } 10

select Sum(student\_mark) from student  $\Rightarrow$  10

select (Sum(student\_mark)) as (mark) from student

old name

new name

select max(student\_mark) from student  $\Rightarrow$  Max(4)

" min ( " ) "  $\Rightarrow$  min (1)

" " avg ( " ) from "  $\Rightarrow$  avg ( )

Select <sup>count</sup> (student\_name) from student  $\Rightarrow$  ans (4)

We can use where

Select count (student\_name) from student where  
department = "CSF".  
(ans: 2)

SQL Having clause

| Employee ID | first name | Lastname | Department | Salary |
|-------------|------------|----------|------------|--------|
| 1           | John       | Doe      | HR         | 55000  |
| 2           | Jane       | Smith    | IT         | 60000  |
| 3           | Bob        | Johnson  | IT         | 62000  |
| 4           | Alice      | Williams | HR         | 54000  |
| 5           | Eva        | Davis    | Finance    | 58000  |
| 6           | Nice       | Brown    | Finance    | 59000  |

Select <sup>count</sup> (\*) from employee where department = "IT"  
 $\rightarrow$  ans: 2.

Select count (\*) from employee group by department  
 $\rightarrow$  ans: department wise count  $\begin{pmatrix} 2 \\ 2 \\ 2 \end{pmatrix}$

Select count (\*), department from employee  
group by department

$\rightarrow$  ans:

| count | department |
|-------|------------|
| 2     | HR         |
| 2     | IT         |
| 2     | Finance    |

[Select avg(salary), department from employee  
group by department]

ans:

| avg (salary) | department |
|--------------|------------|
| 54500.000    | HR         |
| 61000.000    | IT         |
| 58500.000    | Finance    |

After group by we can't use (where)

Whenever we want to set a condition based  
on the aggregate function we can  
use (HAVING).

avg salary below 60k matlu print pannanu

Select avg(salary), department from employee  
group by department having avg(salary) < 60000

ans:

| avg (salary) | department |
|--------------|------------|
| 54500.000    | HR         |
| 58500.000    | Finance    |

1). Find departments with an avg salary greater than  
55000 or have more than two Employees

Select avg(salary), count(\*), department from  
employee group by department having  
avg(salary) > 55000 or count(\*) > 2.

ans:

| avg (salary) | count(*) | department |
|--------------|----------|------------|
| 61000.000    | 2        | IT         |
| 58500.000    | 2        | Finance    |

## Constraint

It is the rule for a database to describe which data is to be allowed (or) not allowed.

### Main constraints in SQL

Primary Key, foreign key, Not null, unique, check, Default.

#### Not Null:

```
Create Table Student {  
    id int,  
    name Varchar (100) Not null  
};
```

O/P: must Be if  
          |                  |  
          Anun              null ? x  
                              ↓  
                              Values must

#### Unique:

Only once not duplicate.

(ex: email) create table student (  
                                  id int,  
                                  email Varchar (100) unique  
                                  );

Primary Key : (One table ke One primary key Haa)

To make each & every rows as unique.

ex: create table student (  
          student\_id int primary key,  
          name Varchar (100);  
          )

[ 25 25 row is  
unique identity ]

#### 2 Rule:

Without duplicate  
without null.

## Foreign Key :

Key Used to connect one table to another.

Table 1 : Department  
dept\_id      dept\_name

10      CSE  
20      ECE

primary  
key

Table 2 : Student

stud\_id      name      dept\_id  
1      Arun      10  
2      Ravi      20  
3      Priya      10

foreign  
key

refers another table  
column.

SQL cmd

```
create table student (  
    student_id int primary key ,  
    name varchar (50),  
    dept_id int foreign key (dept_id) references  
        Department (dept_id)  
);
```

check : (condition must be satisfy)

```
create table student (  
    id int ,  
    age int check (age >= 18)  
);
```

if user enter 17  
it will be  
rejected.

Default : (If the user doesn't give value  
automatically a default value assigned)

```
create table student (  
    id int ,  
    city varchar (30) Default 'Chennai'  
);
```

## Student Table

| student_id | email       | phone | name   |
|------------|-------------|-------|--------|
| 101        | ranun@gmail | 1234  | Anun   |
| 102        | dharama     | 4567  | Dhano  |
| 103        | malini@     | 7890  | malini |

Here :   
 Super key candidate key   
 { Student\_id  $\Rightarrow$  unique   
 email  $\Rightarrow$  unique   
 phone  $\Rightarrow$  unique   
 name  $\Rightarrow$  unique }   
 One will be a primary key   
 remaining keys Alternate keys

Super Key : unique combination of all keys is Super Key   
 max combinations also Super Key.

Like : Student\_id + email  $\rightarrow$  unique.

## Candidate Key :

Same as Super Key  $\rightarrow$  minimal combination

## Composite Key :

Enrollment Table

| Student_id | course_id | marks |
|------------|-----------|-------|
| 101        | 501       | 90    |
| 101        | 502       | 85    |
| 102        | 501       | 80    |

duplicate { 101, 101, 102 }   
 duplicate { 501, 502, 501 }   
 $\rightarrow$  unique

Student\_id + course\_id

= composite

## Join : (consider 2 Table)

Left table  $\Rightarrow$  student

Right table  $\Rightarrow$  Department

| Id | name   | dept_ID |
|----|--------|---------|
| 1  | arun   | 10      |
| 2  | malini | 20      |
| 3  | Dhano  | 10      |
| 4  | raj    | 30      |

| dept_id | dept_name |
|---------|-----------|
| 10      | CSE       |
| 20      | ECE       |
| 40      | AIDS      |

student . dept\_ID   
 department . dept\_id } common column.

Inner Join ( 2 Table layu common aa irukuvathu )

output :

| Name   | dept - name |
|--------|-------------|
| arun   | CSE         |
| malini | ECE         |
| dhanoo | CSE         |

Left JOIN : ( Left Table la irukuvathu kandipa O/P la irukanum )

| <u>Left la</u> |                  | <u>Right la</u> |                                           |
|----------------|------------------|-----------------|-------------------------------------------|
| arun           | } must be in O/P | CSE             | } dept match aana O/P la vacuum (or) NULL |
| malini         |                  | ECE             |                                           |
| dhanoo         |                  | AIDS            |                                           |
| raj            |                  |                 |                                           |

Output :

| Name   | dept - name |
|--------|-------------|
| Arun   | CSE         |
| malini | ECE         |
| dhanoo | CSE         |
| raj    | NULL        |

Right JOIN : ( opposite to left )

O/P:

| name   | dept - name |
|--------|-------------|
| Arun   | CSE         |
| malini | ECE         |
| dhanoo | CSE         |
| Null   | AIDS        |

Full Outer Join : ( Left Join + Right join )

| <u>Left la</u> |  | <u>Right la</u> |
|----------------|--|-----------------|
| arun           |  | CSE             |
| malini         |  | ECE             |
| dhanoo         |  | AIDS            |
| raj            |  |                 |

O/P:

|        |      |
|--------|------|
| arun   | CSE  |
| malini | ECE  |
| priya  | CSE  |
| raj    | null |
|        | AIDS |